



OAKLAND COMMUNITY COLLEGE®

ROBOTICS/AUTOMATED SYSTEMS TECHNOLOGY

ROB 2140 Siemens LADDER LOGIC LABS

John Sefcovic
Oakland Community College

TABLE OF CONTENTS

SIEMENS LADDER LOGIC LABS

LAB: MOVE INSTRUCTION, PART 1	1
LAB: MOVE INSTRUCTION, PART 2	5
LAB: TIMED PROCESS OUTPUT	13
LAB: TEMPERATURE CONVERSION with INDICATORS	22
LAB: ANALOG INPUT	31

Recommend Add-On Instruction First

© 2025 John Sefcovic and Oakland Community College. This work is intended for educational purposes. The author and publisher make no express or implied warranty of any kind, nor shall they be liable in any event for any type of damage or injury, in connection with or arising out of these materials. Siemens TIA Portal is a registered trademarks of Siemens Corporation of Washington, D.C.. All such names are used only in an editorial fashion to the benefit of the trademark owner without any intention of trademark infringements. This work may be reproduced and redistributed, in whole or in part, without alteration and without prior written permission, solely by educational institutions for nonprofit administrative or educational purposes provided all copies contain this copyright statement and the Oakland Community College logo. This work is reproduced and distributed with the permission of John Sefcovic. No other use is permitted without the express prior written permission of John Sefcovic or Oakland Community College.

LAB: MOVE INSTRUCTION, PART 1

PROGRAM FILE: MOVE {last name} {first name}

OBJECTIVE: Manipulating integer data. Use programming to change the preset of a timer instruction.

PART 1 must be submitted first, then PART2.

The lab is in two parts. Create PART 1 first.

NO CREDIT WILL BE GIVEN IF PART 1 IS NOT SUBMITTED FIRST.

After completing PART1, modify for PART 2, print, and submit.

DO NOT DOWNLOAD OR USE THE HMI, THE HMI IS FOR PART 2. SEE PAGE 4.

PART 1 - Create the Initial Program

THE MOVE INSTRUCTION MUST BE USED TO CHANGE THE TIMER PRESET.

BEFORE STARTING TO CREATE THE PROGRAM, SEE NOTES ON PAGE 3.

The operator has three preset time values, 10, 15, or 20 seconds, that can be selected depending on the type of part being produced on the machine. The operator turns a three-position selector switch, making the contact to change the preset value. The operator then presses and releases the START pushbutton, and the process output will remain on for the time period selected.

PLC Tags

INPUTS:

Address	PLC IO Tag	Description
I0.0	SEL_SW_1	Selector Switch, Position 1
I0.1	SEL_SW_2	Selector Switch, Position 2
I0.2	SEL_SW_3	Selector Switch, Position 3
I0.4	START	START Momentary N. O. Pushbutton

OUTPUT:

Address	PLC IO Tag	Description
Q.0.0	OUTPUT	Process Output

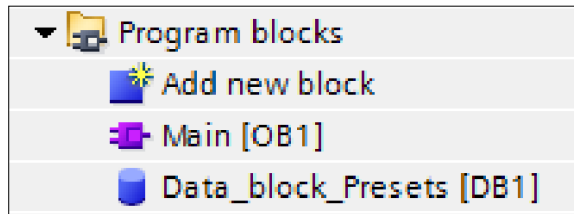


Program Structure

Global Data Blocks

[Data_block_Presets](#)

Create the Global Data Block.



Name of Data Block: [Data_block_Presets](#).

DATA BLOCK:

Data Block Tag	Data Type	Descriptions
PRE_SET_1	Integer	Preset No. 1
PRE_SET_2	Integer	Preset No. 2
PRE_SET_3	Integer	Preset No. 3
ONE_SHOT	Array[0..2] of Bool	One Shot Control

When the Timer is added to the program, rename the Timer's IEC_Block.

Name of System Data Block: [DWELL](#).

Created when Timer instruction added to the program.

SYSTEM BLOCK TIMER:

Data Block Tag	Data Type	Description <i>Cannot be entered as Description</i>
DWELL	IEC_Timer	Process Timer
DWELL.IN*	Timer Boolean	Process Timer Enabled
DWELL.Q*	Timer Boolean	Process Timer Complete

* Created automatically in DWELL timer structure.

continue



NOTES:

Set the value for the PRE_SET_n tag.

Start value: Value that the tag assumed at startup. The value can be changed by program instructions.

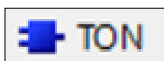
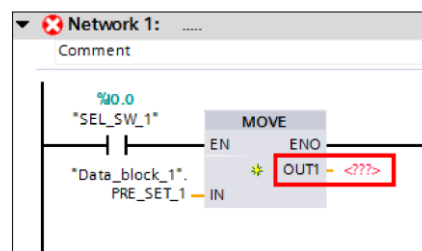
Data_block_Presets				
	Name	Data type	Start value	Retain
1	Static			☐
2	PRE_SET_1	Int	10000	☒
3	PRE_SET_2	Int	15000	☒
4	PRE_SET_3	Int	20000	☒

The default values defined in a code block are used as start values during the creation of the data block.

Retain: Marks the tag as retentive. The values of retentive tags are retained even after the power lost.

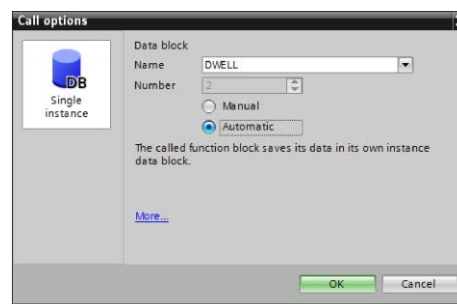
When entering the MOVE instructions, the OUT1 for the timer's preset, PT, cannot be entered until after the Timer Data Block is created.

Create the Timer Network first, see below, then go back and add the timer's .PT to the three MOVE instructions OUT.



When the TON block instruction is dragged on to the network rung, a dialog popup appears.

Change the **Name:** to DWELL.



DOWNLOAD

Download the program to the PLC.

Select the **Monitor** icon for the PLC program.



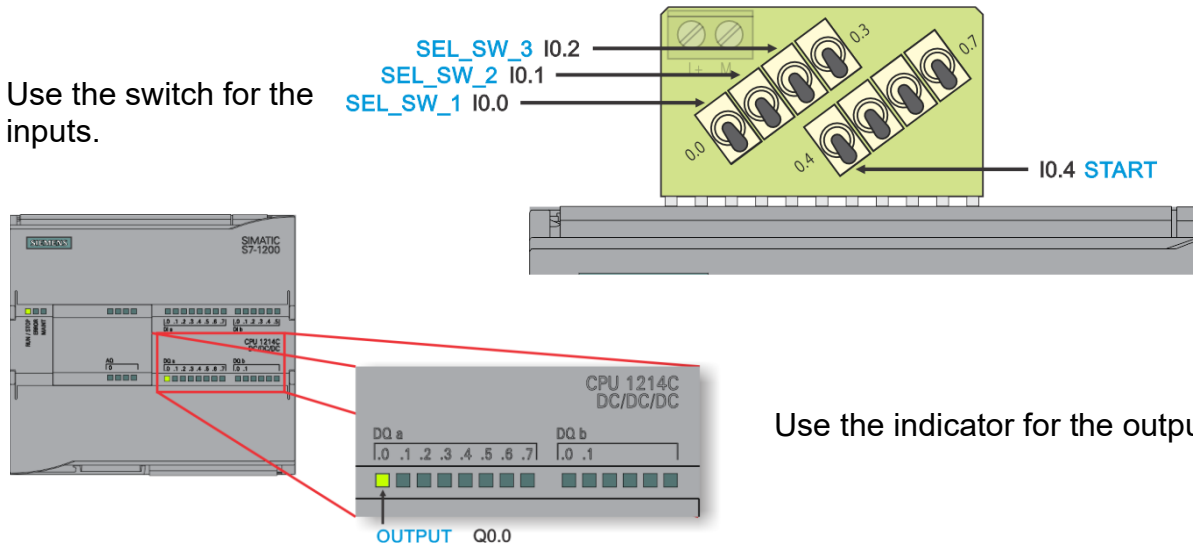
DO NOT DOWNLOAD OR USE THE HMI, THE HMI IS FOR PART 2.

continue



TESTING

Use the switch for the inputs.



-
1. Push up a **switch I0.0** SEL_SW_1 to select a Preset.

Note: Only one Selector Switch (SEL_SW_n) should be up at a time.

2. Push the **switch I0.4** START switch up.
3. Push the **switch I0.4** START switch down.

The output indicator for the **OUTPUT** status should be ON for 10 seconds.

4. Push down a **switch I0.0** SEL_SW_1.

-
1. Push up a **switch I0.1** SEL_SW_2 to select a Preset.

Note: Only one Selector Switch (SEL_SW_n) should be up at a time.

2. Push the **switch I0.4** START switch up.
3. Push the **switch I0.4** START switch down.

The output indicator for the **OUTPUT** status should be ON for 15 seconds.

4. Push down a **switch I0.1** SEL_SW_2.

-
1. Push up a **switch I0.2** SEL_SW_3 to select a Preset.

Note: Only one Selector Switch (SEL_SW_n) should be up at a time.

2. Push the **switch I0.4** START switch up.
3. Push the **switch I0.4** START switch down.

The output indicator for the **OUTPUT** status should be ON for 10 seconds.

4. Push down a **switch I0.2** SEL_SW_3.



LAB: MOVE INSTRUCTION, PART 2

PROGRAM FILE: MOVE *created for PART 1*

OBJECTIVE: Manipulating integer data. Use programming to change the preset of a timer instruction. Adding logic considerations for HMI

PART 2 - Add Function routine and Modifications for HMI

Program Structure

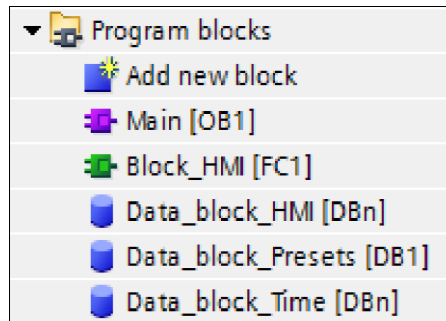
Program Function routines:

[Block_HMI](#)

Global Data Blocks:

[Data_block_HMI](#)

[Data_block_Time](#)



n = automatically assigned block number

Create the Global Data Blocks and rename. Add the following Tags.

Name of Data Block: [Data_block_HMI](#)

DATA BLOCK:

Data Block Tag	Data Type	Descriptions
HMI_START	Boolean	HMI START Pushbutton
HMI_PB	Integer	Operator Panel Pushbuttons
HMI_PB_1	Boolean	Operator Panel Pushbutton 1
HMI_PB_2	Boolean	Operator Panel Pushbutton 2
HMI_PB_3	Boolean	Operator Panel Pushbutton 3
HMI_ENABLE_DISABLE	Boolean	Allows Hardwired Panel
HMI_PT	Real	HMI Timer Preset
HMI_ET	Real	HMI Timer Elapsed



Name of System Data Block: **Data_block_Time**.

DATA BLOCK:

Data Block Tag	Data Type	Descriptions
PT_DINT	Double Integer	Timer PT Converted to Integer
PT_REAL	Real	Timer PT Converted to Real
ET_DINT	Double Integer	Timer ET Converted to Integer
ET_REAL	Real	Timer ET Converted to Real

LOGIC FOR FUNCTION ROUTINE HMI

Name of Function: **Block_HMI**

The values are selected via the Operator's Human Machine Interface (HMI) panels. A START button on the HMI turns on the process output for the selected time period. The current preset and accumulated values are displayed. A second panel allows the three preset values to be entered.

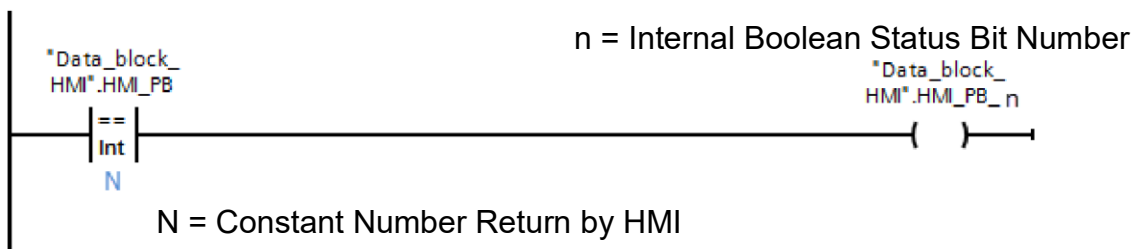
As a backup a hardwired operators panel can be used when not disabled by the HMI.

In a Function routine create the rungs:

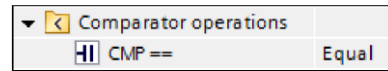
The OP_PB tag returns the value of the Operator Pushbutton pressed on the HMI.

The EQU instructions sets correspondingly, the status bit OP_PB_1 when OP_PB value is 1, set status bit OP_PB_2 when OP_PB value is 2, and set status bit OP_PB_3 when OP_PB value is 3.

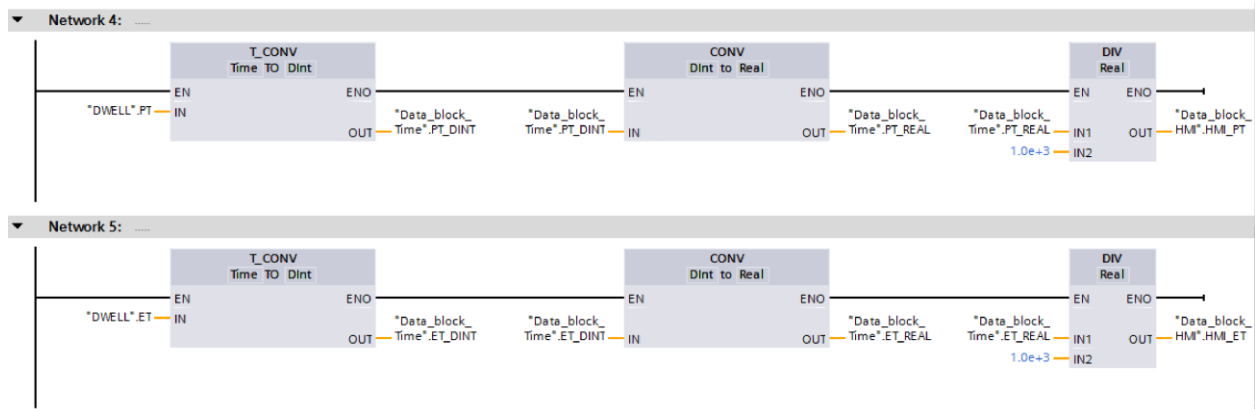
Three similar rungs are needed.



The **EQUAL** instruction is found in the **Compare** operations.



Use Convert Instructions and an Arithmetic Instruction to change the preset and elapsed dwell timer's values to represent the preset and elapsed into a Real value for seconds.

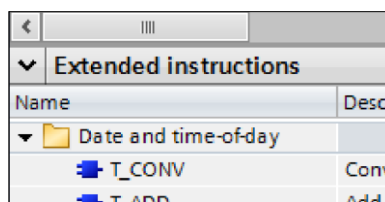


Additional tags for the converted values are in the Data Block **Data_block_Time**.

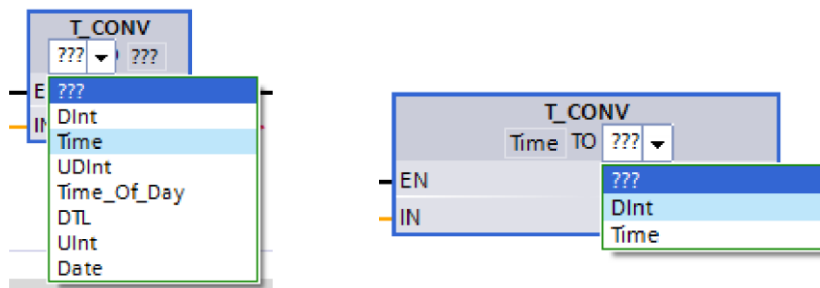
The Timer's Preset and Elapsed values are in a Day, Hour, Minute, and Second format and must be converted to a 32 bit Double Integer.

Date and time-of-day

Use the **T_CONV** instruction. The instructions is found in the **Extended instructions** under **Date and time-of-day**.



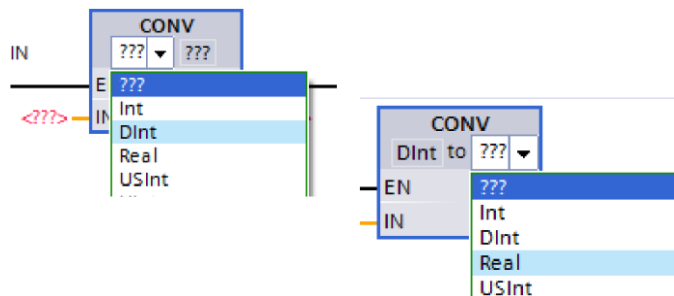
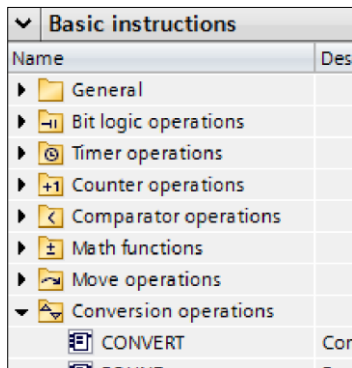
The type of data to be converted must be selected. **Time to DInt**.



Conversion operations

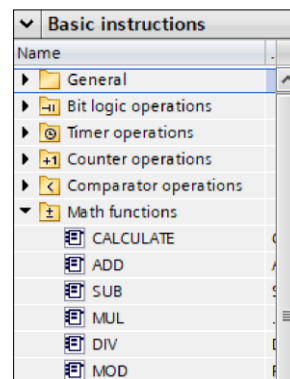
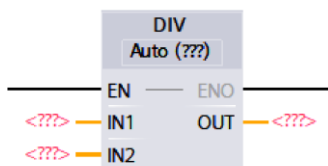
The CONV instruction then converts the Double Integer to a Real value for use in the Divide Instruction. The instructions is found in the **Basic instructions Conversion operations**.

The type of data to be converted must be selected. DInt to Real.



Math Operations

The Divide DIV instruction is found in the **Basic instructions Math functions**.



continue



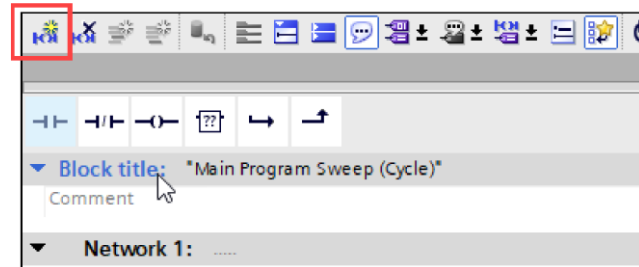
MODIFICATIONS OF MAIN ORGANIZATION BLOCK

Modification 1

Add a new Network 1.

Select the text **Title Block:** above the existing Network 1.

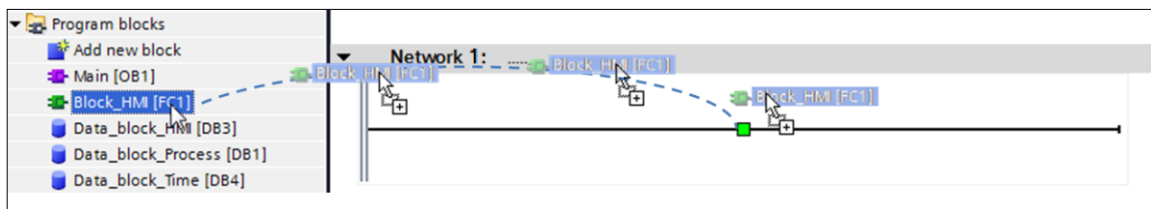
Then, select the **Insert Network** icon to add the Network.



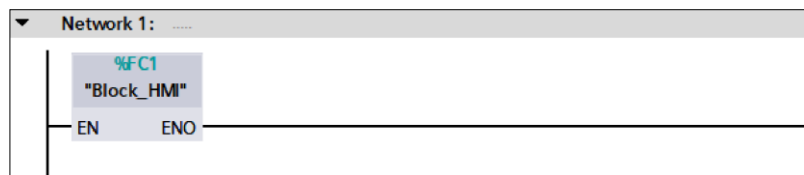
Modification 2

 The HMI Function routine must be created first.

Drag from the Project Tree the **Function** routine **Block_HMI** onto the last rung of the **Organization Block**.



The **Function** routine **Block_HMI** is called unconditionally.

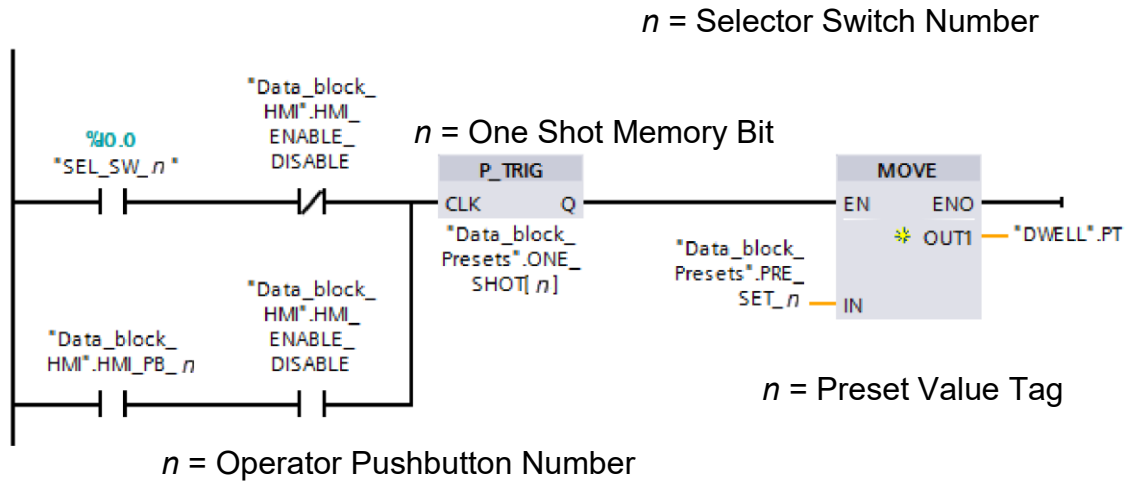


Modification 3

Modify the switch rungs to move the selected preset value with either the Selector Switch or the Operators Panel.

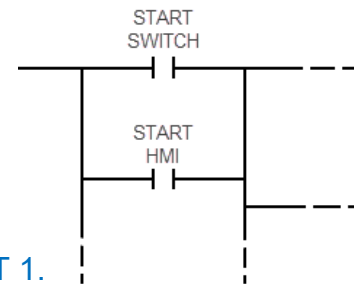
The selector switch will only be operational if the HMI_ENABLE_DISABLE on the operator's panel is low.

Note - Information Only: This condition HMI_ENABLE_DISABLE SWITCH low would exist if the HMI panel was not operational.



Modification 4

Add the HMI_START from the HMI pushbutton.



Keep the holding circuit from PART 1.

DOWNLOAD

Download the program to the PLC.

Select the **Monitor** icon for the PLC program.



*For each opened Program the **Monitor** icon **must be** selected.*

Download the HMI program.

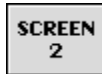
continue



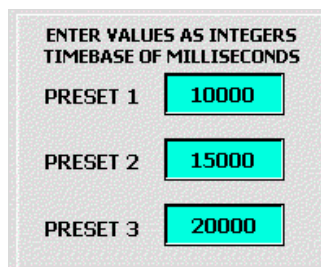
TESTING

ENTER TIMER PRESETS IN MILLISECONDS

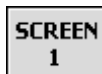
1. Select the **SCREEN 2** button.



2. Enter, in millisecond, a 10, 15, and 20 second Timer Presets.

A screen titled "ENTER VALUES AS INTEGERS TIMEBASE OF MILLISECONDS". It contains three rows: "PRESET 1" with a value of "10000", "PRESET 2" with a value of "15000", and "PRESET 3" with a value of "20000". Each value is displayed in a red box.

3. Select the **SCREEN 1** button to return to the Main screen.

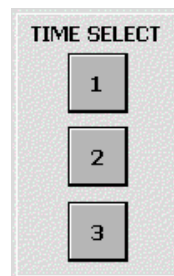


ENABLING AND SELECT TIME VALUE

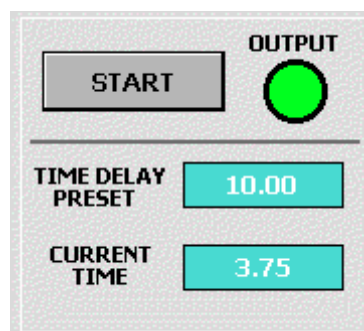
1. Check for the HMI to be **ENABLED**.



2. Select the **1** button.
3. **TIME DELAY PRESET** should read 10.00.
4. Press the HMI **START** button.



The **OUTPUT** should be on while the **CURRENT TIME** is elapsing.



continue



5. Select **2** and press the **START** button for the **OUTPUT** to be ON for 15 seconds.
6. Select **3** and press the **START** button for the **OUTPUT** to be ON for 20 seconds.

TESTING THE HMI ENABLE\DISABLE

1. Check for the HMI to be **ENABLED**.



2. Push up a **switch I0.0 SEL_SW_1**. The **TIME DELAY PRESET** should not change.
3. Push down a **switch I0.0 SEL_SW_1**.
4. Push up a **switch I0.1 SEL_SW_2**. The **TIME DELAY PRESET** should not change.
5. Push down a **switch I0.1 SEL_SW_2**.
6. Select the **1** button. **TIME DELAY PRESET** should read 10.00.
7. Push up a **switch I0.2 SEL_SW_3**. The **TIME DELAY PRESET** should not change.
8. Push down a **switch I0.2 SEL_SW_3**.

-
1. **DISABLE** the HMI.



2. Select the **2** button. The **TIME DELAY PRESET** should not change.
 3. Select the **3** button. The **TIME DELAY PRESET** should not change.
 4. Push up a **switch I0.2 SEL_SW_3**. The **TIME DELAY PRESET** should be 20.00.
 5. Push down a **switch I0.2 SEL_SW_3**.
 6. Select the **1** button. The **TIME DELAY PRESET** should not change.
-

LAB: TIMED PROCESS OUTPUT

PROGRAM FILE: PROCESS_OUTPUT {last name} {first name}

OBJECTIVE: Arithmetic instructions and Floating Point data type.

Create a program for a process that uses multiple layers of a sheet material. The type and number of sheets loaded depends on the order. The operator enters one of four types of sheet materials that will be used and the number of sheet layers. Only one type of material is used at a time, and up to six sheets may be used.

The operator presses a HMI panel pushbutton that corresponds to the material type, then presses a HMI panel pushbutton for the number of sheets. When the operator presses and releases the START pushbutton, a process output will turn on for the computing determined time value based on the type and number of sheets. The following parameters are used to then control the process timer.

Material A - multiplier of 0.73 times the number of sheets

Material B - multiplier of 1.05 times the number of sheets

Material C - multiplier of 1.23 times the number of sheets

Material D - multiplier of 1.86 times the number of sheets

Notes on Programming:

The calculated process time will need to be multiplied by 1000 for milliseconds for the timer preset to use.

PLC Tags

INPUTS:

Address	PLC I/O Tag	Descriptions
I.0.0	SW_START	START Momentary N. O. Pushbutton

OUTPUT:

Address	PLC I/O Tag	Descriptions
Q0.0	PROCESS_OUT	Process Output

Process output = Type * + of sheets

continue



Program Structure

Program Function routines:

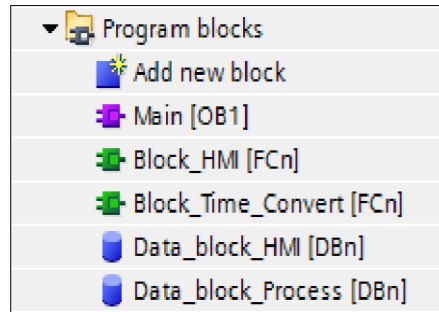
[Block_HMI](#)

[Block_Time_Convert](#)

Global Data Blocks:

[Data_block_HMI](#)

[Data_block_Process](#)



n = automatically assigned block number

Create a Global Data Block and rename. Add the following Tags.

Name of Data Block: [Data_block_Process](#)

DATA BLOCK:

	Data Block Tag	Data Type	Descriptions
A thru D	A_MATERIAL_MULTIPLIER	Real	Material A Multiplier
	B_MATERIAL_MULTIPLIER	Real	Material B Multiplier
	C_MATERIAL_MULTIPLIER	Real	Material C Multiplier
	D_MATERIAL_MULTIPLIER	Real	Material D Multiplier
Selected Value	MATERIAL_MULTI_VALUE	Real	Material Multiplier Value
	PROCESS_VALUE	Real	Process Value

Type * + of sheets

Name of Data Block: [Data_block_HMI](#)

DATA BLOCK:

	Data Block Tag	Data Type	Descriptions
INPUTS	HMI_TYPE_SHEET	Integer	Operator Panel Pushbuttons Value for Sheet Type
	HMI_NUM_SHEET	Integer	Operator Panel Pushbuttons Value for Number of Sheets



OUTPUTS Calculated

INPUT

HMI_PT_PROCESS_TIME	Real	Operator Panel Process Preset Time Value for program Timer Preset.
HMI_ET_PROCESS_TIME	Real	Operator Panel Process Elapsed Time Value for Program Timer Elapsed.
HMI_SW_START	Boolean	HMI Start Switch

LOGIC FOR FUNCTION ROUTINE TIME CONVERT

Create the Function routine and rename.

Name of Function: **Block_Time_Convert**

Add to the **Function Time_Convert** Interface the Input, Output, and Temp tags.

INTERFACE for FUNCTION - Block_Time_Convert:

Input Tag	Data Type	Descriptions
IEC_TIMER_PT	Time	Timer PT Converted to Integer
IEC_TIMER_ET	Time	Timer PT Converted to Real

Output Tag	Data Type	Descriptions
IEC_TIMER_REAL_PT	Real	Timer ET Converted to Integer
IEC_TIMER_REAL_ET	Real	Timer ET Converted to Real

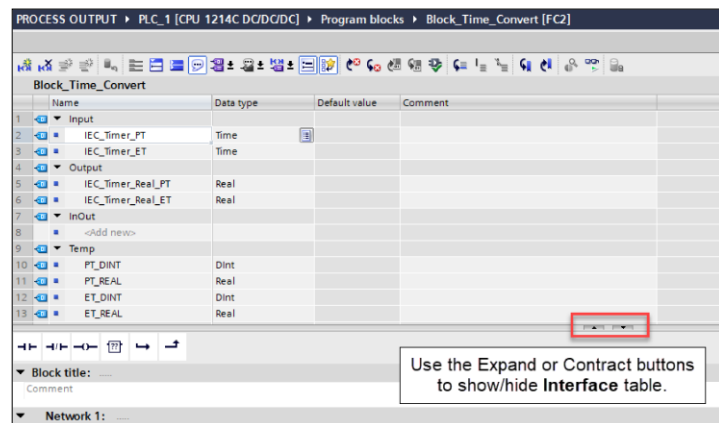
Temp Tag	Data Type	Descriptions
PT_DINT	Double Integer	Preset Value as Integer
PT_REAL	Real	Preset Value as Real
ET_DINT	Double Integer	Elapsed Value as Integer
ET_REAL	Real	Elapsed Value as Real

continue for INTERFACE illustration



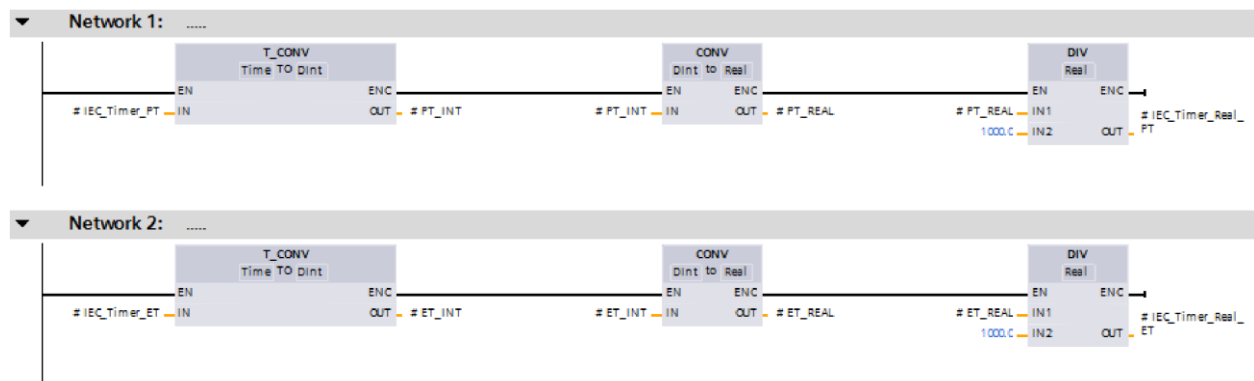
The Interface will create the Function's Local Tags to pass the Timer's Preset and Elapsed Value into the Function and (**Input Tags**) return out (**Output Tags**) the converted Preset and Elapsed time value for the HMI.

Local Tags (**Temp**) will hold the intermediate values used for the conversions.



Use Convert Instructions and an Arithmetic Instruction to change the preset and elapsed dwell timer's values to represent the preset and elapsed into a Real value for seconds.

The Timer's Preset and Elapsing values as Time Data Types are converted to Real values for displaying in the HMI in units of *seconds*.



LOGIC FOR FUNCTION ROUTINE HMI

Name of Function: **Block_HMI**

Four similar Networks use a Compare instruction determines the four values of the HMI_SHEET_TYPE to be equal to HMI Interlock button value and moves the selected A_MATERIAL_MULTIPLIER, B_MATERIAL_MULTIPLIER, C_MATERIAL_MULTIPLIER, or D_MATERIAL_MULTIPLIER into the MATERIAL_MULTI_VALUE.

continue

Network 5, of this program is for the Function call **Block_Time_Convert**.

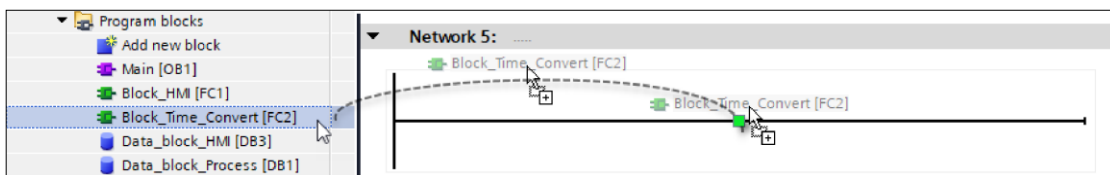
 The System IEC Timer Block must be created first in the ORGANIZATION BLOCK.

Return to add this network rung after adding the timer to the Organization Block Main, page 17, or create the IEC Timer Block manually.

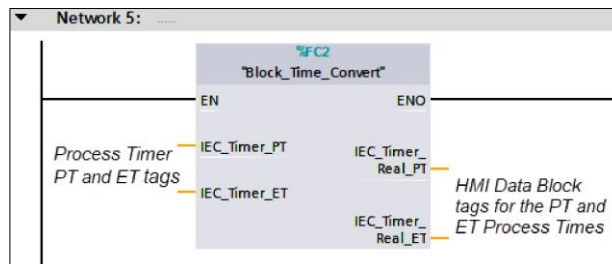
In this Function call of **Block_Time_Convert** for the HMI, the called Function converts the Timer's values to Real data to display on the HMI.

The Function when called passes the Timer's Preset and Elapsing values in to the routine's logic. Passed out of routine are the Preset and Elapsing values as Real Data Types.

Drag from the Project Tree the **Function** routine **Block_Time_Convert** onto the last rung of Block_HMI.



Only have one timer in the program. Easier than adding many timers use one block



Real Number Display on the HMI. Need create timer first

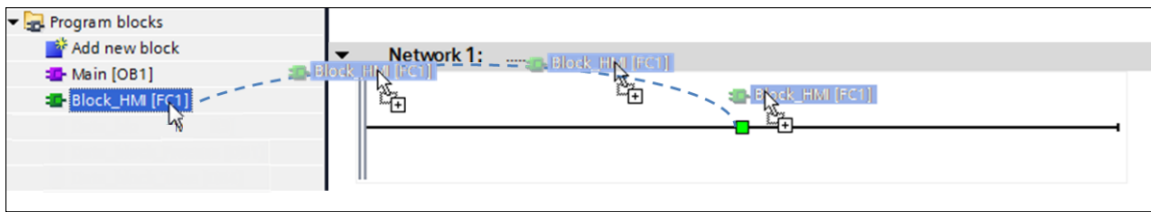
LOGIC FOR MAIN ORGANIZATION BLOCK:

 The HMI Function routine must be created first.

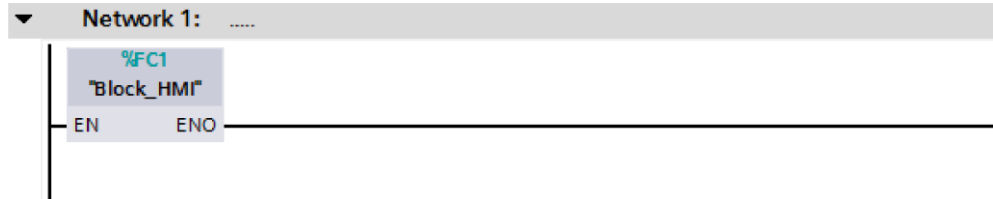
Drag from the Project Tree the **Function** routine **Block_HMI** onto the first rung of the **Organization Block**. The **Function** routine **Block_HMI** is called unconditionally.

Continue for FUNCTION drag illustrations





The Function for the HMI is called unconditionally.



Network Logic Descriptions

The pressing of either the START pushbutton or HMI START calculates by using a Multiply instruction for the product the result `PROCESS_VALUE` of the `HMI_NUM_SHEET` and `MATERIAL_MULTI_VALUE`.

A second Multiply instruction, controller by the START or HMI START, converts the `PROCESS_VALUE` to milliseconds for Preset Time of the Timer, *see next page for the CONV instruction*.

[Parallel the start see handout](#)

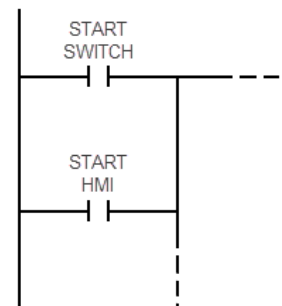
The pressing of either the START pushbutton or HMI START then turns on the Output and begins the timer elapsing for the Process Value time. To then turn off the Output at the completion of the timing cycle

Siemens does not allow the assignment of a hardware address, such as I0.0, to a HMI input objects. The write, input bit, can only be from a non-hardware data source.

By using the hardwired switch or the HMI switch in the parallel logic structure, the START can be initiated from either source, the switch or HMI.

In the Network for Main OB, the `HMI_SW_START` must be in parallel with the hardwired `SW_START` for the calculation Network and the process output Network.

Example shown is a generic logic structure.
[Will require a holding circuit.](#)



continue



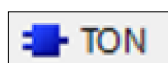
Name of System Data Block: **PROCESS_TIMER**

SYSTEM BLOCK TIMER: *Created when Timer instruction is added to the program.*

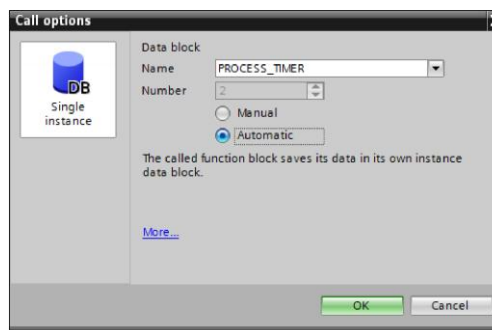
Data Block Tag	Data Type	Descriptions
PROCESS_TIMER	IEC_Timer	Process Timer
PROCESS_TIMER.IN*	Timer Boolean	Process Timer Enabled
PROCESS_TIMER.Q*	Timer Boolean	Process Timer Complete

* Created automatically in DWELL timer structure.

Organization Block Timer Instruction.



When the TON block instruction is dragged on to the network rung, a dialog popup appears.

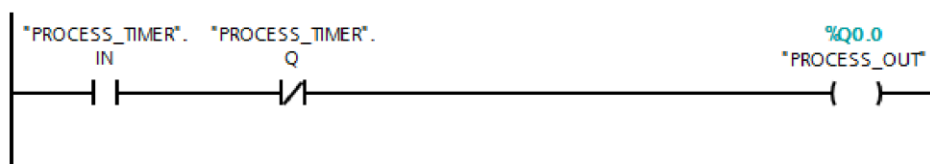


Change the **Name:** to PROCESS_TIMER .



Return to Function: Block_HMI and complete Network 5. See page 15.

There is no equivalent to the Timer Timing Bit, the Timers IN (Enabled) bit and Timer's Output (Done) bit must be used for the timed output.

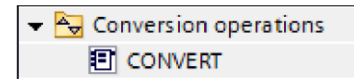


The second multiple instruction must be defined as a Real to produce the PROCESS_VALUE. A timer accepts **Dint** data type for the Preset Time.

The Convert (**CONV**) instruction will change the Real data type to a Dint (double integer) without truncating or rounding.

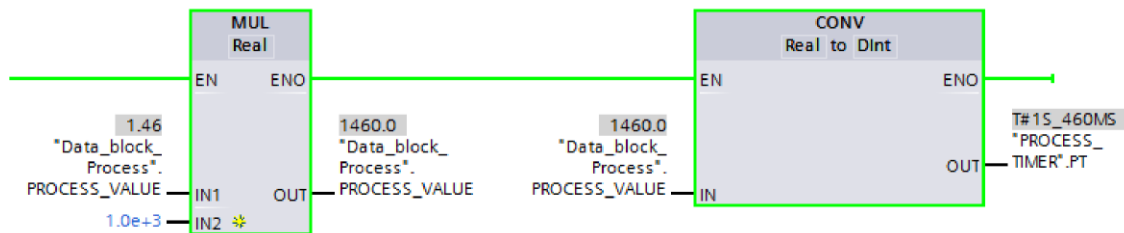
The convert's Out to the timer's preset formats the date a Time data type.

The Convert (**CONV**) instruction is in the Conversion operations folder.



See below for logic example.

Get rid of decimal real before timer.



DOWNLOAD

Download the program to the PLC.

Select the **Monitor** icon for the PLC program.



*For each opened Program the **Monitor** icon **must be** selected.*

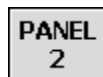
Download the HMI program.

TESTING

ENTERING MATERIAL MULTIPLIER

Do this first or else all zeros!

1. Press the button to go to **PANEL 2**.



2. Enter the values for the **A**, **B**, **C**, and **D MATERIALS**.

MULTIPLIER VALUE SET-UP	
MATERIAL A	0.73
MATERIAL B	1.05
MATERIAL C	1.23
MATERIAL D	1.86

3. Press the button to return to the **MAIN** screen.



SELECTING SHEET TYPE AND NUMBER

1. Press the Button for **TYPE B**.
2. Press the Button for **NUMBER 2**.

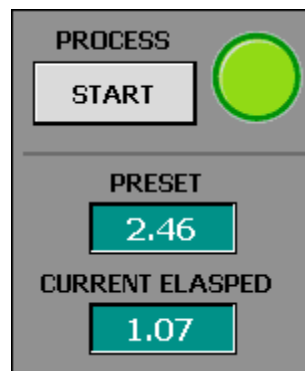
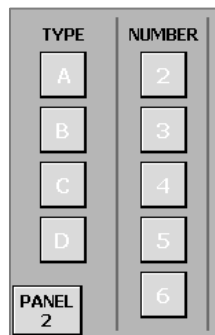
$$1.05 \times 2 = 2.10$$

3. Press the **START** button.



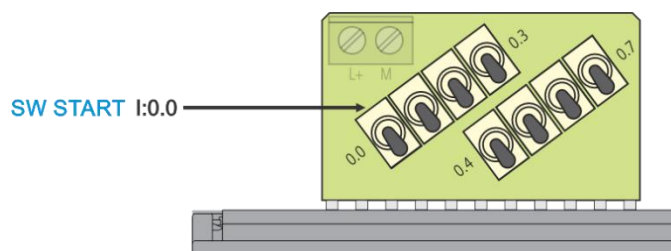
The **OUTPUT** should be ON while the timer is **ELASPING**.

$$1.23 \times 2 = 2.46$$



4. Select the other **TY PE** and **NUMBER** buttons and then **START** to verify the different values calculated values for the Timer's **PRESET**.

TESTING THE START SWITCH



1. Push up a **switch I0.0** SW START.
2. The **OUTPUT** should be ON while the timer is **ELASPING**
3. Push down a **switch I0.0** SW START.

Separate Data Blocks

LAB: TEMPERATURE CONVERSION with INDICATORS

PROGRAM FILE: CONVERT {last name} {first name}

OBJECTIVE: Comparison instructions, Binary Coded Decimal data type, and the buffering of data. Comparison instruction to determine a data range

Create a program that will convert a Celsius temperature to a Fahrenheit temperature value. The maximum Celsius value will be 100 degrees.

The project will use either hardwired or HMI input pushbuttons. A maintenance screen will set a signal to disable the use of the hardwired pushbuttons.

PLC Tags

INPUT:

Address	PLC I/O Tag	Descriptions
I0.0	CONVERT_PB	Convert Temperature N. O. pushbutton
I0.3	RESET_PB	Data Error Reset, N. O. pushbutton

OUTPUT:

Address	PLC I/O Tag	Descriptions
Q0.0	ERROR_DUP_VAL	Error Indicator
Q0.1	ERROR_OVER_VAL	Data Error, Over Value
Q0.2	LL_INDC	Low Low Indicator
Q0.3	L_INDC	Low Indicator
Q0.4	M_INDC	Mid Indicator
Q0.5	H_INDC	High Indicator
Q0.6	HH_INDC	High High Indicator

continue



Different from 5K separate data blocks

Program Structure

Program Function routine:

Block_Indicators

Program Function Block routines:

Block_Input_Control

Block_Convert

Function Blocks Data Blocks below
will be created when Function Block
call is added to the program.

Function Block Data Blocks:

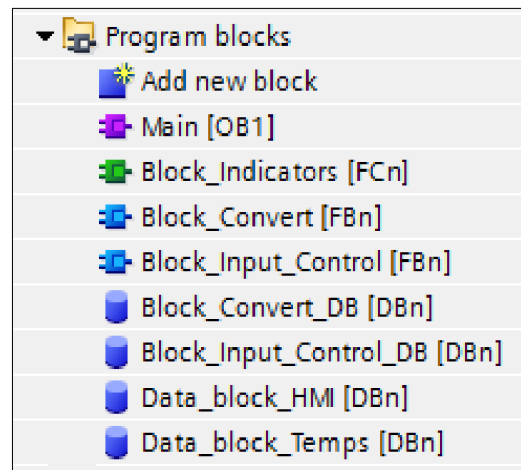
Block_Convert_DB

Block_Input_Control_DB

Global Data Blocks:

Data_block_HMI

Data_block_Temps



n = automatically assigned block number

Create the Global Data Blocks and rename. Add the following Tags.

Name of Data Block: **Data_block_HMI**

DATA BLOCK:

Data Block Tag	Data Type	Descriptions
HMI_CONVERT_PB	Boolean	HMI Convert Button
HMI_RESET_PB	Boolean	HMI Reset Button
HMI_ENABLE_DISABLE	Boolean	HMI Enable/Disable Button

Name of Data Block: **Data_block_Temps**

DATA BLOCK:

Data Block Tag	Data Type	Descriptions
C_TEMP	DINT	Input of Celsius Temperature



C_TEMP_STORAGE	DINT	Storage of Celsius Temperature
F_TEMP	DINT	Converted Fahrenheit Result

LOGIC FOR FUNCTION BLOCK CONVERT

Name of Function Block: **Block_Convert** **Function Block**

You must use the **individual Math functions** instructions to convert the Celsius value to a Fahrenheit value.

$$\text{Equation: } F = (C(9)/5) + 32$$

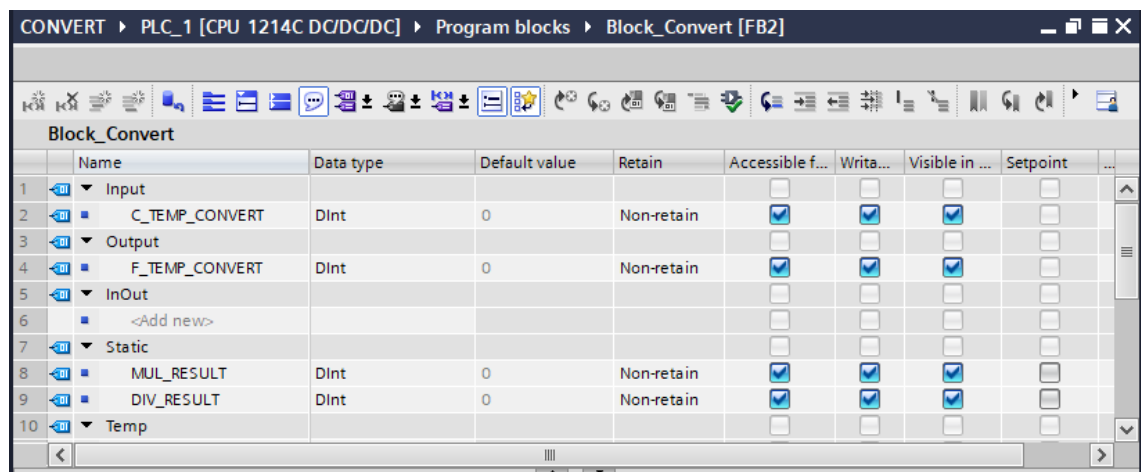
INTERFACE for Function Block - Block_Convert:

Input Tag	Data Type	Description
C_TEMP_CONVERT	DINT	Celsius Temperature to Function Block

Output Tag	Data Type	Description
F_TEMP_CONVERT	DINT	Fahrenheit Temperature from Function Block

Static Tag	Data Type	Descriptions
MUL_RESULT	DINT	Multiply Result
DIV_RESULT	DINT	Divide Result

*Interface for the Function Block **Block_Convert**.*



LOGIC FOR FUNCTION BLOCK INPUT_CONTROL

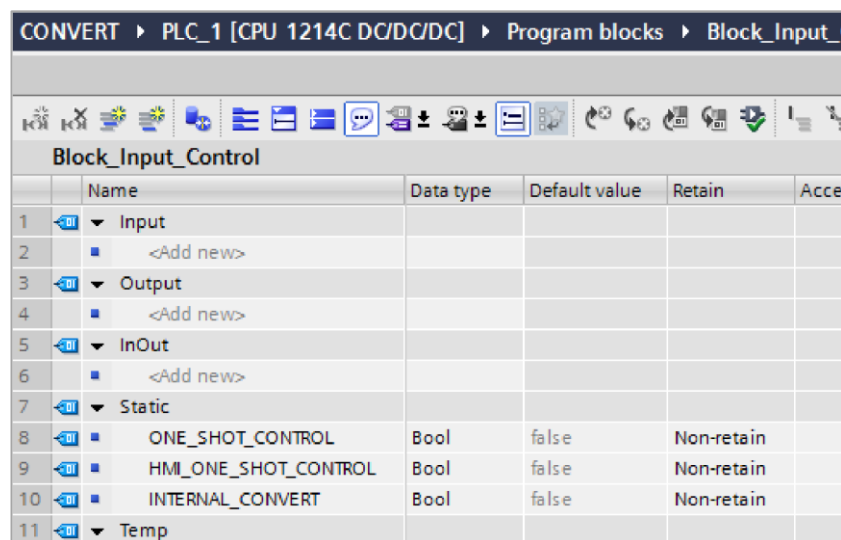
Name of Function Block: **Block_Input_Control**

NOTE: For the one-shot instruction, --|P|--, Scan Operand for Positive Signal Edge.

INTERFACE for Block Input Control:

Interface Static	Data Type	Descriptions
ONE_SHOT_CONTROL	Boolean	One Shot, Hardwired Panel
HMI_ONE_SHOT_CONTROL	Boolean	One Shot, HMI
INTERNAL_CONVERT	Boolean	Internal Control, Enable Convert

*Interface for the Function Block **Block_Input_Control**.*



	Name	Data type	Default value	Retain	Access
1	▼ Input				
2	■ <Add new>				
3	▼ Output				
4	■ <Add new>				
5	▼ InOut				
6	■ <Add new>				
7	▼ Static				
8	■ ONE_SHOT_CONTROL	Bool	false	Non-retain	
9	■ HMI_ONE_SHOT_CONTROL	Bool	false	Non-retain	
10	■ INTERNAL_CONVERT	Bool	false	Non-retain	
11	▼ Temp				

Note: The CONVERT TEMPERATURE and RESET pushbuttons (switches) will only be operational if HMI ENABLE DISABLE is reset (logic 0).

When the operator presses the CONVERT TEMPERATURE HMI, Pushbutton or the Hardwired Pushbutton (switch), the program will perform the following error checks.

If the value entered has not changed, a Duplicate Error indicator will be lit.

A check for the input temperature is 100 degrees or less will be performed. If above 100 degrees, turn on Over Value indicator will be lit.

Any error will prevent a conversion of the input value.



The operator must then press the HMI Pushbutton or the Hardwired Pushbutton (switch) for RESET and enter the correct value.

If there are no errors, duplicate and overvalue, with convert signal the Function Block Convert is called.

LOGIC FOR FUNCTION INDICATORS

The **In Range** instruction for the range indicators *must* be used.

Name of Function: **Block_Indicators**

If a conversion is valid, indicators will show the converted temperature is within a LOW LOW, LOW, MID, HIGH, or HIGH HIGH range. Use the IN_RANGE instructions.

Ranges:	Low Low	32 to 49
	Low	50 to 99
	Mid	100 to 149
	High	150 to 199
	High High	200 to 212

LOGIC FOR MAIN ORGANIZATION BLOCK

The Function Block **Block_Input_Control** for the conversion is called unconditionally.

The Function **Block_Indicators** for the indicators is called unconditionally.

DOWNLOAD

Download the program to the PLC.

Select the **Monitor** icon for the PLC program.



*For each opened Program the **Monitor** icon *must* be selected.*

Download the HMI program.

continue

TESTING

DISABLE HARDWARE PANEL

1. From the MAIN HMI screen select the **MAINTENANCE** panel.



2. Select **HARDWIRED PANEL DISABLED.**

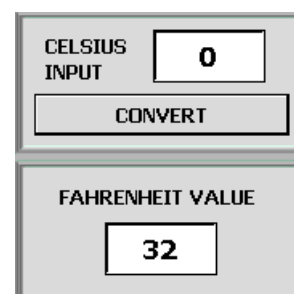


TESTING TEMPERATURE CONVERSION

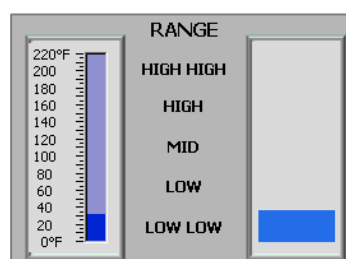
1. From the **MAINTENANCE** screen go to the **INPUT** screen.



2. Enter a Celsius value of 0°.
3. Toggle (ON-OFF) the **CONVERT** pushbutton
4. Check for the correct converted temperature, *should be 32° F.*

The screen shows two sections. The top section is labeled "CELSIUS INPUT" and contains a text box with the number "0". Below this is a "CONVERT" button. The bottom section is labeled "FAHRENHEIT VALUE" and contains a text box with the number "32".

5. Go to the **RANGE** screen and check for the correct **Range Indicator.**



6. Go to the **INPUT** screen.



Repeat the steps 4, 5, and 6 with the other Celsius values into C_TEMP to test the Range Indicators.

Celsius	Fahrenheit	Range
0°	32°	Low Low
30°	86°	Low
60°	140°	Mid
80°	176°	High
100°	212°	High High

continue

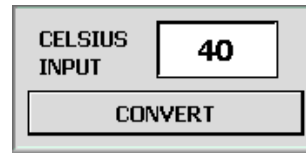


TEST DUPLICATE ERROR

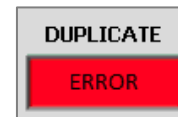
1. Go to the **INPUT** screen.



2. Enter a Celsius value of 40.
3. Press the **CONVERT** pushbutton.
4. Again, Press the **CONVERT** pushbutton.



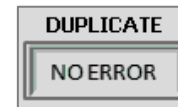
ERROR DUPLICATE VALUE indicator should be ON.



Toggle (ON-OFF) the **RESET** pushbutton.

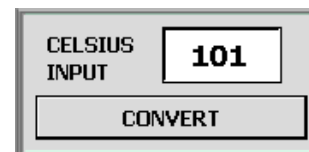


ERROR DUPLICATE VALUE indicator should be OFF.

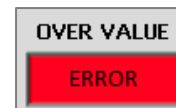


TEST OVER TEMPRATURE VALUE ERROR

1. Enter a Celsius value of 101°.
2. Toggle (ON-OFF) the **CONVERT** pushbutton.



ERROR OVER VALUE indicator should be ON.



3. Toggle (ON-OFF) the **RESET** pushbutton.



ERROR OVER VALUE indicator should be OFF.



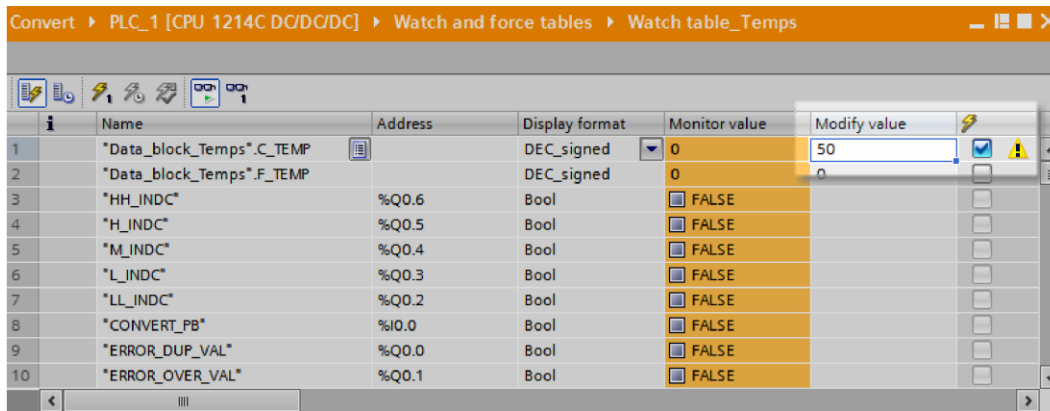
continue



Once Online - you can set it up for a time later or create a parameter that if something occurs make the change. we select modify once icon

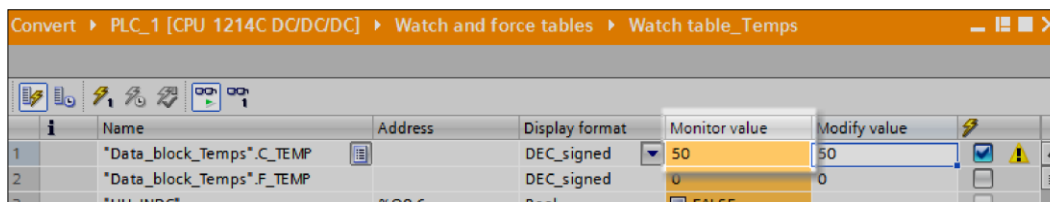
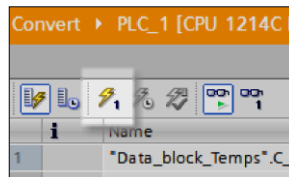
INFORMATION - TO CHANGE A TAG VALUE WITH THE WATCH WINDOW

Type in a value in the **Modify value** for "Data_block_Temps".C_TEMP.



Select the **Modify once** icon.

The value in the Data Tag has changed.



ENABLE HARDWIRED PANEL

1. Go to the **MAINTENANCE** screen.



2. Select **HARDWIRED PANEL ENABLED**.

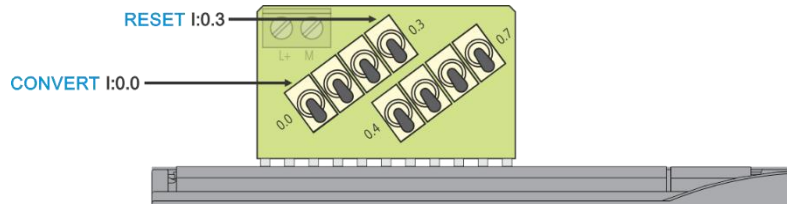


MODIFY A VALUE AND USE THE SWITCHES

1. Set "Data_block_Temps".C_TEMP to a value of 50 in the **Watch Window**.

continue



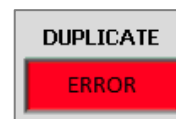


2. Turn the **switch I0.0** CONVERT to the **up** position.
3. Turn the **switch I0.0** CONVERT_PB to the **down** position.
4. Check for the correct converted temperature, **should be 122° F**.

TEST DUPLICATE ERROR

1. Do Not Change the "**Data_block_Temps**".**C_TEMP** value.
2. Turn the **switch I0.0** CONVERT to the **up** position.
3. Turn the **switch I0.0** CONVERT to the **down** position.

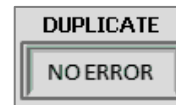
DUPLICATE VALUE indicator should be **ON**.



RESET

1. Turn the **switch I0.3** RESET to the **up** position.
2. Turn the **switch I0.3** RESET to the **down** position.

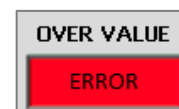
ERROR OVER VALUE indicator should be **OFF**.



TEST OVER VALUE ERROR

1. Set "**Data_block_Temps**".**C_TEMP** to a value of 101 in the **Watch Window**.
2. Turn the **switch I0.0** CONVERT to the **up** position.
3. Turn the **switch I0.0** CONVERT to the **down** position.

ERROR OVER VALUE indicator should be **ON**.



-
1. Turn the **switch I0.3** RESET to the **up** position.
 2. Turn the **switch I0.3** RESET to the **down** position.

ERROR OVER VALUE indicator should be **OFF**.



LAB: ANALOG INPUT

PROCESS MODEL: ANALOG_INPUT {last name} {first name}

OBJECTIVE: Analog input for process control.

The analog input is used to measure the height of thin stamped metal parts as they are feed to an automatic assembly machine. Duplicate parts are assembled as a stacked “sandwich” before being joined. The stamped parts are picked by vacuum cups off of a stack and placed onto a conveyor. The single analog sensor measures the height of the part to make sure that only one part was placed on the conveyor.

If two parts are stuck together, the height measurement will detect the two parts. The process is stopped, and a Feed Error indicator will turn on. This indicator stays on until the Reset pushbutton is pressed.

NOTE: The Feed Error must be latched (set) on to retain status, after problem corrected and before reset, during power failure, or change of modes.

The operator will correct the problem, and then press Reset to turn off the Error indicator. The Start pushbutton is pressed to restart the system.

Above 45% two parts

Two proximity switches are used to detect the part type. The analog sensor will then measure the height of the part. This is not an actual calibrated measurement, but a relative measurement based on the percentage of movement of the linear stroke of the sensor. Part A has a nominal percentage of 30, a value above 45% will indicate two parts. Part B has a nominal percentage of 40, a value above 55% will indicate two parts.

Above 55% will be two parts

Each Part has a unique feature - photoelectric senses detects it

INPUTS:

Address	PLC I/O Tag	Descriptions
I0.0	PART_A	Part A N. O. Proximity Switch
I0.1	PART_B	Part B N. O. Proximity Switch
I0.2	RESET	RESET N. O. Pushbutton
I0.4	START	START N. O. Pushbutton
I0.7	STOP	STOP N. C. Pushbutton

continue



OUTPUTS:

Address	PLC I/O Tag	Descriptions
Q0.0	CONVEYOR	Conveyor Motor
Q0.1	ERROR	Part Feed Error Indicator

Pre-created **Simulated Analog Input [DBn]** Data Block.

ANALOG INPUT: Use simulated Analog Input

Tag	Data Block	Descriptions
HEIGHT_AI	Simulated Analog Input	Height Sensor

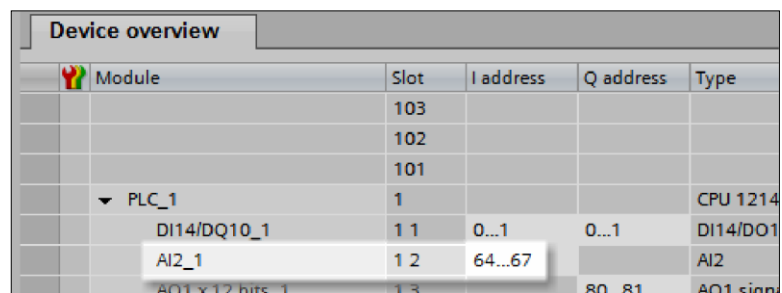
PRECREATED - FOR US

This program will use a simulated analog input, that is, a Data Block is used to hold in Integer value for the analog signal.

INFORMATION ONLY:

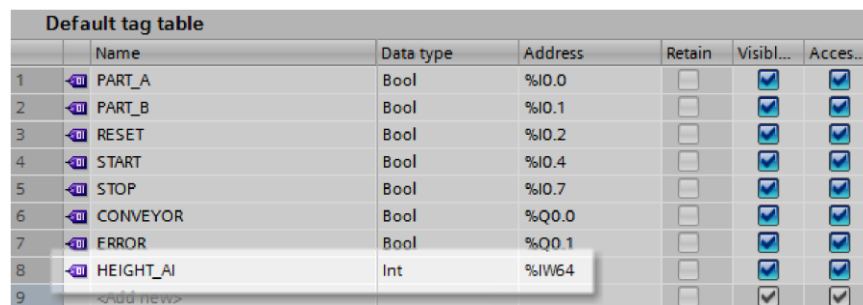
The PLC has two Analog Inputs.

For this processor the actual analog input address is Byte 64 and Byte 65 for 16 Bits.



Device overview				
Module	Slot	I address	Q address	Type
	103			
	102			
	101			
PLC_1	1			CPU 1214
DI14/DQ10_1	1 1	0...1	0...1	DI14/DO1
AI2_1	1 2	64...67		AI2
AO1 x 12 bits_1	1 3		80...81	AO1 sign

The tag assignment **IW** (Input **W**ord) is an **Integer** Data Type with the address **%IW64**.



Default tag table						
	Name	Data type	Address	Retain	Visibl...	Acces...
1	PART_A	Bool	%I0.0	☐	☒	☒
2	PART_B	Bool	%I0.1	☐	☒	☒
3	RESET	Bool	%I0.2	☐	☒	☒
4	START	Bool	%I0.4	☐	☒	☒
5	STOP	Bool	%I0.7	☐	☒	☒
6	CONVEYOR	Bool	%Q0.0	☐	☒	☒
7	ERROR	Bool	%Q0.1	☐	☒	☒
8	HEIGHT_AI	Int	%IW64	☐	☒	☒
9	<add new>			☐	☒	☒

for input w for word

There is no Analog Input attached to the PLC, this lab assignment will use **HEIGHT_AI** Tag in pre-created **Simulated Analog Input [DBn]** Data Block.



Program Structure

Program Function routine:

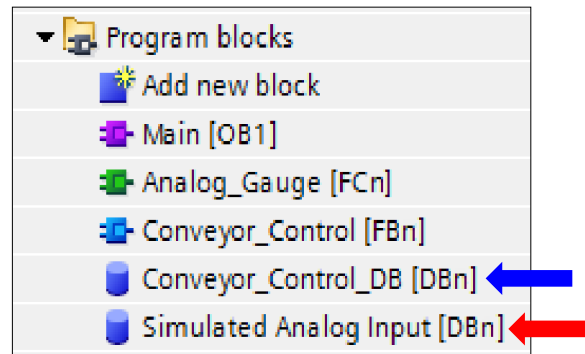
[Analog_Gauge](#)

Program Function Block routine:

[Conveyor Control](#)

Function Block Data Block

[Conveyor_Control_DB](#)



n = automatically assigned block number

[Conveyor_Control_DB](#) is created automatically when call to the Function Block ([Conveyor Control](#)) is dragged into the program.

The Data Block **Simulated Analog Input** has been pre-created and is used in the HMI to enter the analog bit value.

LOGIC FOR FUNCTION ANALOG GAUGE

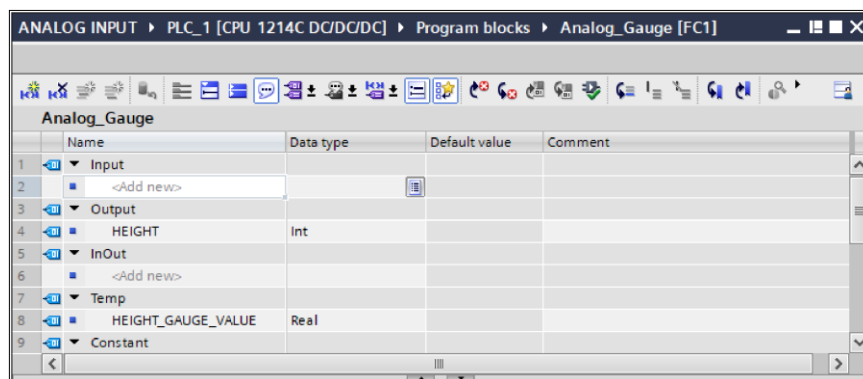
Name of Function: [Analog_Gauge](#)

INTERFACE for Function - [Analog_Gauge](#):

Output Tag	Data Type	Description
HEIGHT	INT	Scaled Analog Value

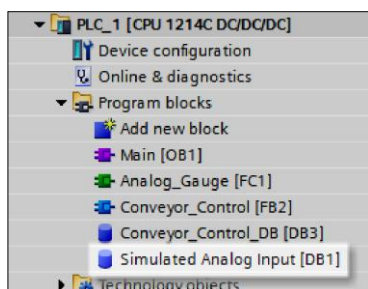
Temp Tag	Data Type	Descriptions
HEIGHT_GAUGE_VALUE	REAL	Normalized Analog Value

Between 0 inputed value
is the simulated value.
-> Output of normalized input
to the scale x



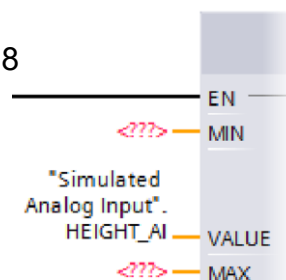
Use the instructions to Normalize and Scale the integer value from the analog input. The Scaled Value is 0 to 100.

The instructions are in **Basic Instructions** folder **Conversion operations**.



Analog Minimum Bit Value: 0
Analog Maximum Bit Value: 27648

Use the tag **HEIGHT_AI** from the Data Block **Simulated Analog Input [DBn]**.



LOGIC FOR FUNCTION BLOCK CONVEYOR

Name of Function: **Conveyor_Control**

INTERFACE for FUNCTION BLOCK - Conveyor_Control:

Input Tag	Data Type	Descriptions
HEIGHT	Integer	Passed Analog value

Static Tag	Data Type	Descriptions
PART_A_ERROR	Boolean	Part A double part
PART_B_ERROR	Boolean	Part B double part
PART_A_UPPER	INT	Part A Upper Limit
PART_B_UPPER	INT	Part B Upper Limit

INTERFACE for FUNCTION BLOCK - Conveyor_Control

	Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...
1	Input						
2	HEIGHT	Int	0	Non-ret...	☒	☒	☒
3	Output						
4	<Add new>						
5	InOut						
6	<Add new>						
7	Static						
8	PART_A_ERROR	Bool	false	Non-retain	☒	☒	☒
9	PART_B_ERROR	Bool	false	Non-retain	☒	☒	☒
10	PART_A_UPPER	Int	45	Retain	☒	☒	☒
11	PART_B_UPPER	Int	55	Retain	☒	☒	☒
12	Temp						



DATA BLOCK for FUNCTION BLOCK - Conveyor_Control

Conveyor_Control							
	Name	Data type	Default value	Retain	Accessible f...	Visible in ...	Setpoint
7	Static				☐	☐	☐
8	PART_A_ERROR	Bool	false	Non-retain	☒	☒	☐
9	PART_B_ERROR	Bool	false	Non-retain	☒	☒	☐
10	PART_A_UPPER	Int	45	Retain	☒	☒	☒
11	PART_B_UPPER	Int	55	Retain	☒	☒	☒

When entering Upper Part values, assign as Integer Data Type, Retain, and Setpoint.

Retain - value is retained on power loss.

Setpoint - allow setpoint initial value snapshot to be restored.

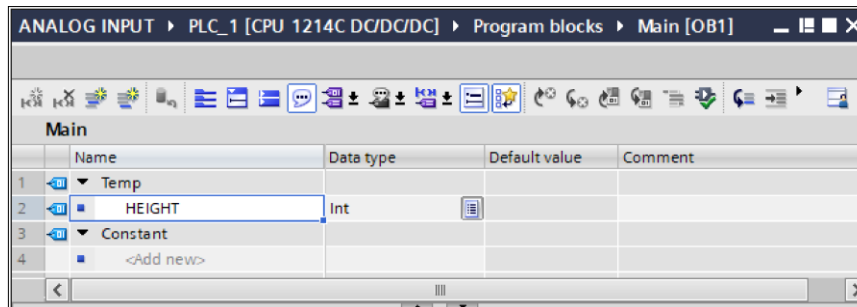
See above explanation for logic requirements.

LOGIC FOR MAIN ORGANIZATION BLOCK

INTERFACE for ORGANIZATION BLOCK - Main:

Temp Tag	Data Type	Descriptions
HEIGHT	INT	Passed Analog value

INTERFACE for FUNCTION BLOCK - Main



The Function **Analog_Gauge** is called unconditionally.

The Function Block **Conveyor_Control** is called unconditionally.

When prompted for the Function Block's Data Block, keep the default name, **Conveyor_Control_DB [DBn]**.

continue



DOWNLOAD

Download the program to the PLC.

Select the **Monitor** icon for the PLC program.



*For each opened Program the **Monitor** icon **must be** selected.*

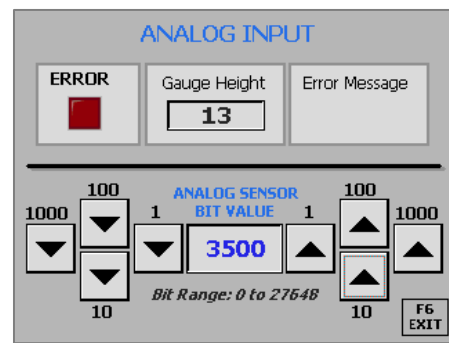
Download the HMI program.

TESTING

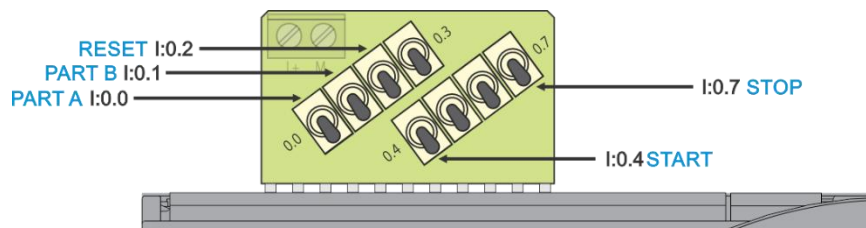
INFORMATION - ANALOG SENSOR BIT VALUES

Use the **UP** or **DOWN** arrows to set the Bit Value for the Input sensor.

The Scaled Value is the **Gauge Height**.



START CONVEYOR



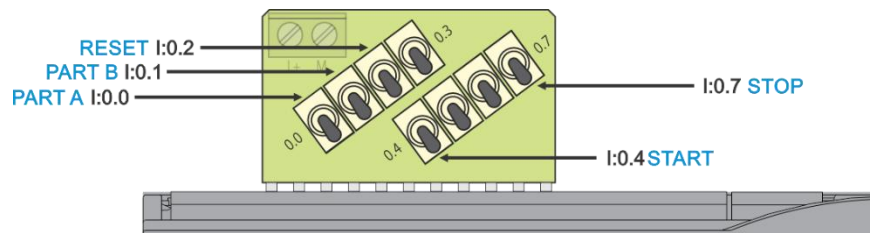
1. Push up **switch I0.7 STOP**.
2. Push up **switch I0.4 START**.

In the PLC program, the CONVEYOR Output should be ON.

3. Push down **switch I0.4 START**.

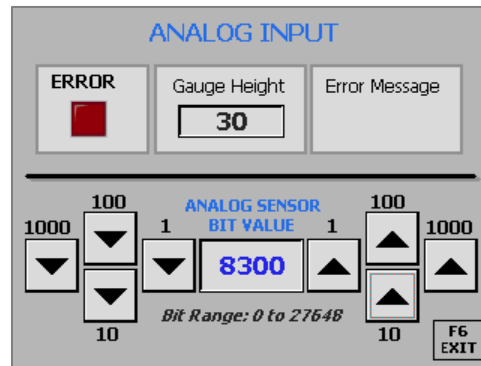
continue





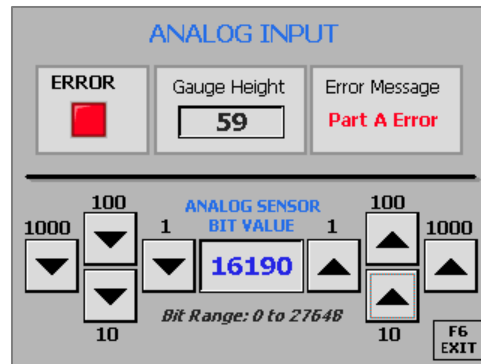
TESTING FOR A SINGLE PART A

1. Use the **UP** or **DOWN** arrows to set the Bit Value of the Input sensor for a single Part A.
2. Push up **switch I0.0 PART A**.
There should be no Error.
3. Push down **switch I0.0 PART A**.



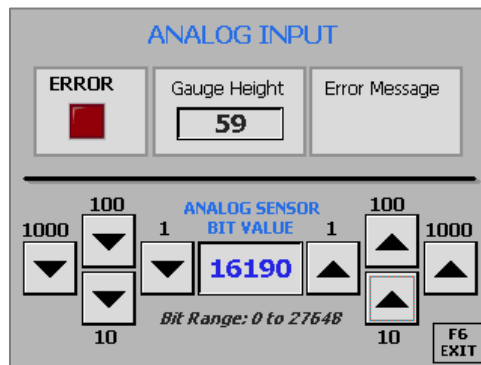
TESTING FOR A DOUBLE PART A

1. Use the **UP** or **DOWN** arrows to set the Bit Value of the Input sensor for a double Part A.
2. Push up **switch I0.0 PART A**.
There should be an Error.
Conveyor should be OFF.
3. Push down **switch I0.0 PART A**.



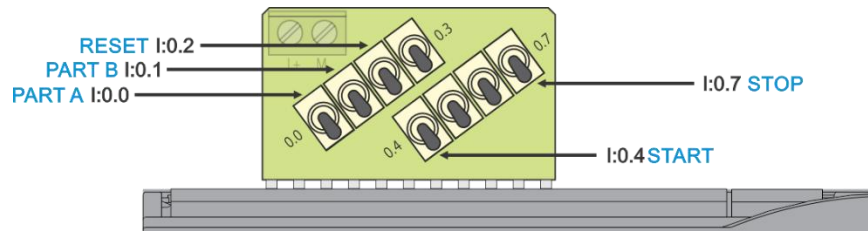
RESET ERROR

1. Push up **switch I0.2 RESET**.
There should be no Error.
2. Push down **switch I0.2 RESET**.



continue





START CONVEYOR

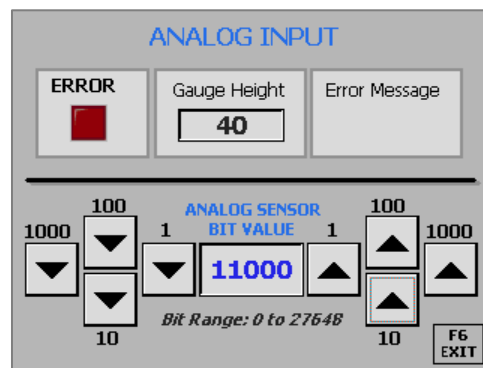
1. Push up **switch I0.4** START.

In the PLC program, the CONVEYOR Output should be ON.

2. Push down **switch I0.4** START.

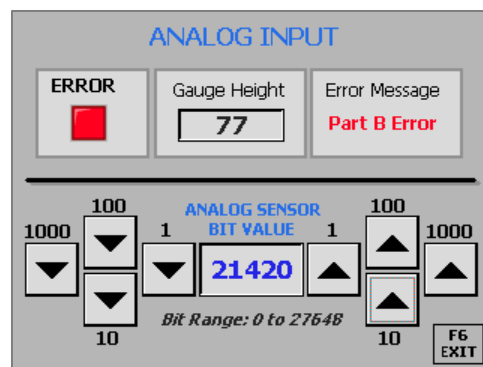
TESTING FOR A SINGLE PART B

1. Use the **UP** or **DOWN** arrows to set the Bit Value of the Input sensor for a single Part B.
2. Push up **switch I0.1** PART B.
There should be no Error.
3. Push down **switch I0.1** PART B.

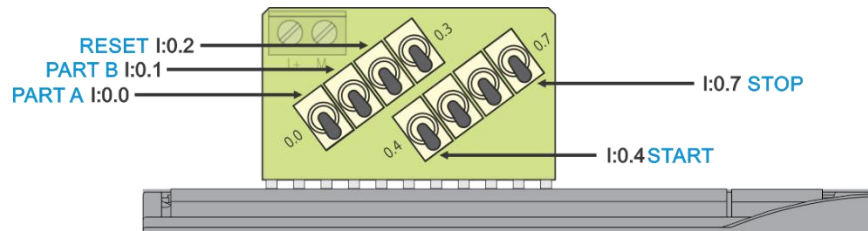


TESTING FOR A DOUBLE PART B

1. Use the **UP** or **DOWN** arrows to set the Bit Value of the Input sensor for a double Part B.
2. Push up **switch I0.1** PART B.
There should be an Error.
Conveyor should be OFF.
3. Push down **switch I0.1** PART B.

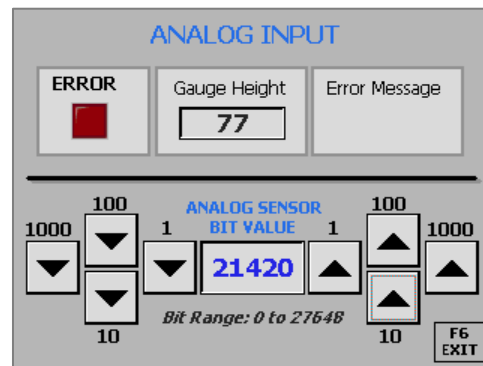


continue



RESET ERROR

1. Push up **switch I0.2** RESET.
There should be no Error.
2. Push down **switch I0.2** RESET.



START CONVEYOR

1. Push up **switch I0.4** START.
- In the PLC program, the CONVEYOR Output should be ON.
2. Push down **switch I0.4** START.

STOP CONVEYOR

1. Push up **switch I0.4** START.
- In the PLC program, the CONVEYOR Output should be ON.
2. Push down **switch I0.4** START.
-

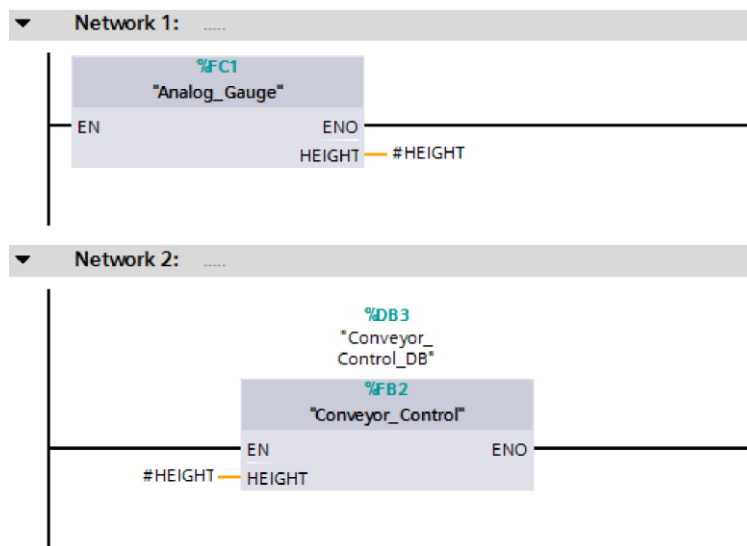
continue

INFORMATION ONLY

Analog_Gauge		
	Name	Data type
1	▼ Input	
2	■ <Add new>	
3	▼ Output	
4	■ HEIGHT	Int
5	▼ InOut	
6	■ <Add new>	
7	▼ Temp	
8	■ HEIGHT_GAGE_VALUE	Real

Analog gauge called height hashtag in front is strictly local.
Used by that block only.

By creating an Output instance **Height** in the **Analog_Gauge** Function...



... the value is temporarily used by the **Main OB1** to pass the value to the Input instance of the Function Block **Conveyor_Control**.

Main		
	Name	Data type
1	▼ Temp	
2	■ HEIGHT	Int
3	▼ Constant	

By using the Local Tags in the **Analog_Gauge** Function and **Main OB1**, the value from analog sensor is “protected” from change by any other programming block.

Notice the #HEIGHT tags begin with the # mark indicating a **Local** Tag.

